# OPERATING SYSTEM (CT 656)

Tribhuvan University | Institute of Engineering | BCT | Year III / Part II

# CHAPTER 8: Security

**Syllabus Hours: 4 hrs | Marks Weight: Part of 20 marks (Chapters 6, 8, 9)**

Complete Exam-Ready Answers | All Past Questions Covered (2073–2082)

## CHAPTER 8 SYLLABUS TOPICS

- 8.1 Security Breaches (Unauthorized access, data leaks, software attacks)
- 8.2 Types of Attacks (Passive, Active, Insider, Outsider, Network Attacks)
- 8.3 Security Policy and Access Control (ACL, Protection Matrix, Capability Lists)
- 8.4 Basics of Cryptography (Symmetric: Caesar/DES, Asymmetric: RSA, Public Key)
- 8.5 Protection Mechanisms (Protection Domain, Object, Rights)
- 8.6 Authentication (Passwords, Biometrics, Two-factor)
- 8.7 OS Design Considerations For Security (Least privilege, Isolation, Minimization)
- 8.8 Access Control Lists And OS Support (UNIX permissions, ACL, Firewalls)

## SECTION 1: Security Breaches, Types of Attacks & Network Security

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2080 Chaitra, 2079 Jestha, 2079 Ashwin, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Magh

### Q1. Explain the types of Network Attacks. What is ACL? Why is the 'HASH' function called a Message Digestor? [2+2+2]

**Exam Ref:** 2082 Kartik, 2081 Chaitra, 2080 Chaitra, 2081 Ashwin, 2079 Jestha, 2079 Ashwin

**Answer:**

### Types of Network Attacks

| Attack Type | Description | Example |
|---|---|---|
| **Passive Attack** | Attacker eavesdrops and monitors data without modifying it. Hard to detect. | Wiretapping, packet sniffing, traffic analysis |
| **Active Attack** | Attacker intercepts and modifies data or creates false data. | Replay attack, modification of messages, DoS |
| **DoS (Denial of Service)** | Floods a server with requests to make it unavailable to legitimate users. | SYN flood, Smurf attack, bandwidth exhaustion |
| **DDoS** | Distributed DoS using a botnet of thousands of compromised machines. | Mirai botnet attacking DNS servers (2016) |
| **Man-in-the-Middle** | Attacker secretly intercepts and relays communication between two parties. | SSL stripping, ARP poisoning |

| | | |
|---|---|---|
| **Phishing** | Fraudulent emails/websites that trick users into revealing credentials. | Fake bank login page, spear phishing |
| **SQL Injection** | Malicious SQL code inserted into input fields to manipulate database. | ' OR '1'='1 bypassing login forms |
| **Buffer Overflow** | Writing data beyond buffer boundary to overwrite adjacent memory. | Stack smashing, heap overflow exploits |
| **Trojan Horse** | Malware disguised as legitimate software. | RAT (Remote Access Trojan), keyloggers |
| **Virus/Worm** | Self-replicating malicious code. Worm spreads without user action. | ILOVEYOU worm, WannaCry ransomware |

## ACL — Access Control List

An Access Control List (ACL) is a data structure that specifies which users or system processes are granted access to objects (files, directories, network resources) and what operations are allowed.

ACL Structure: Each resource has an associated list: (Subject, Rights)

```
File: /etc/passwd
  root     : read, write
  alice    : read
  bob      : no access
```

In UNIX, ACL is implemented as: Owner permissions | Group permissions | Others permissions (rwxrwxrwx)

- Advantage: Fine-grained, per-user control; easy to determine who can access a resource
- Disadvantage: Large ACLs are hard to manage; determining which resources a user can access requires scanning all ACLs

## HASH Function — Why Called Message Digestor

A Hash Function takes an input message of arbitrary length and produces a fixed-length output (digest) called the hash value or message digest.

**Why 'Message Digestor':**

- Like biological digestion converts food into a compact form, hashing compresses a large message into a small, fixed-size digest
- One-way: Given the hash, it is computationally infeasible to reconstruct the original message
- Deterministic: Same input always produces same hash
- Avalanche Effect: Even a 1-bit change in input produces completely different hash

| Algorithm | Output Size | Status | Use Case |
|---|---|---|---|
| **MD5** | 128 bits | Broken (collisions found) | Legacy checksum |
| **SHA-1** | 160 bits | Deprecated | Old SSL certificates |
| **SHA-256** | 256 bits | Secure | Bitcoin, TLS, Digital signatures |
| **SHA-3** | 256/512 bits | Latest standard | High-security applications |

Applications: Password storage, digital signatures, file integrity verification, blockchain.

**Q2. What are the security problems associated with OS? Explain ACL with its use in security. Explain how OS manages security. [2+6]**

**Exam Ref:** 2076 Baisakh, 2079 Ashwin Back

**Answer:**

## Security Problems in Operating Systems

| Security Problem | Explanation |
|---|---|
| **Unauthorized Access** | Users accessing resources without permission — violates confidentiality |
| **Data Breaches** | Sensitive data leaked via malware, poor access control, or insider threats |
| **Privilege Escalation** | Attacker gains higher access rights than intended (user → root/admin) |
| **Malware (Virus/Worm/Trojan)** | Malicious software corrupts data, steals information, or takes control |
| **Buffer Overflow Attacks** | Exploits lack of input validation to execute arbitrary code with system privileges |
| **Trapdoor/Backdoor** | Hidden code left by developer providing unauthorized entry; bypasses security |
| **Race Conditions (TOCTOU)** | Time-of-check-to-time-of-use: exploiting timing gap between security check and use |
| **Covert Channels** | Unauthorized information leakage through side channels (timing, storage) |

## How OS Manages Security

- User Authentication: Verifies identity before granting access (password, biometric, token)
- Access Control: ACL and permission bits determine who can read/write/execute
- Process Isolation: Each process has its own address space; cannot access others' memory
- Kernel Mode vs User Mode: Privileged instructions only in kernel mode; user programs restricted
- Audit Logs: System logs all security-relevant events for later review
- Encryption: File systems (BitLocker, eCryptfs) encrypt data at rest

## SECTION 2: Cryptography — Symmetric & Asymmetric Encryption

Exam Ref: 2082 Kartik, 2080 Ashwin, 2079 Chaitra, 2078 Baishakh, 2075 Bhadra, 2074 Bhadra, 2073 Magh

**Q3. What do you mean by symmetric cryptosystem? Explain Caesar Cipher with a suitable example. How is the protection domain implemented in security? [2+3+2]**

**Exam Ref:** 2078 Baishakh, 2079 Chaitra

**Answer:**

### Symmetric Cryptography

In symmetric cryptography, the same key is used for both encryption and decryption. Both sender and receiver must share this secret key securely beforehand.

| Aspect | Symmetric | Asymmetric |
|---|---|---|
| **Keys** | Single shared secret key | Public key (encrypt) + Private key (decrypt) |
| **Speed** | Very fast — suitable for large data | Slow — suitable for small data/key exchange |
| **Key Distribution** | Difficult — must share key securely | Easy — public key freely shared |
| **Examples** | Caesar, DES, AES, 3DES | RSA, ECC, Diffie-Hellman |
| **Use Cases** | Bulk data encryption, VPNs | Key exchange, digital signatures, SSL/TLS |

### Caesar Cipher — Symmetric Encryption Example

Caesar Cipher is one of the oldest and simplest symmetric encryption techniques. It works by shifting each letter in the plaintext by a fixed number of positions in the alphabet.

**Algorithm: C = (P + K) mod 26  where P = plaintext letter position, K = key (shift), C = ciphertext letter**

Decryption: P = (C - K + 26) mod 26

**Example with Key K = 3:**

| Plaintext | Ciphertext |
|---|---|
| **H E L L O** | K H O O R  (H+3=K, E+3=H, L+3=O, L+3=O, O+3=R) |
| **A T T A C K** | D W W D F N  (A+3=D, T+3=W, T+3=W, A+3=D, C+3=F, K+3=N) |
| **K H O O R** | Decrypt (K=3): K-3=H, H-3=E, O-3=L, O-3=L, R-3=O → HELLO |

Limitation: Only 25 possible keys — easily brute-forced. Vulnerable to frequency analysis. Used today only for educational purposes.

### Protection Domain

A Protection Domain defines the set of objects a process can access and the operations (rights) it can perform on each object.

- Domain = {(Object, Rights)} — e.g., Domain1 = {(File1: read), (File2: read/write), (Printer: print)}
- A process executes in exactly one domain at a time

- Domain switching: A process may switch domains (e.g., when calling a privileged function)

Protection Matrix: A 2D table with domains as rows and objects as columns:

| Domain | File1 | File2 | Printer | Disk |
|---|---|---|---|---|
| **Domain1** | read | read, write | print | — |
| **Domain2** | — | read | print | read, write |
| **Domain3** | read, write, exec | — | — | read |

Implementation: Protection matrix stored as ACL (per column) or Capability List (per row).

---

### Q4. Explain private and public key cryptography. What is the use of ACL? [5+3]

**Exam Ref:** 2075 Bhadra, 2073 Magh

**Answer:**

## Public Key (Asymmetric) Cryptography — RSA

Public key cryptography uses two mathematically related keys — a public key and a private key. What is encrypted with one can only be decrypted with the other.

**RSA Algorithm Concept:**

- Key Generation: Choose large primes p, q. Compute n=p×q. Find e (public exponent) and d (private exponent) such that e×d ≡ 1 (mod φ(n))
- Public Key: (e, n) — shared openly with everyone
- Private Key: (d, n) — kept secret by owner
- Encryption: C = M^e mod n
- Decryption: M = C^d mod n

Digital Signatures: Sender signs with PRIVATE key → receiver verifies with PUBLIC key. Proves authenticity and non-repudiation.

| Operation | Who Uses | Which Key |
|---|---|---|
| **Encrypt for confidentiality** | Sender encrypts | Receiver's PUBLIC key |
| **Decrypt message** | Receiver decrypts | Receiver's PRIVATE key |
| **Sign a document** | Sender signs | Sender's PRIVATE key |
| **Verify signature** | Receiver verifies | Sender's PUBLIC key |

---

## SECTION 3: Authentication, Protection Mechanisms & OS Security Design

Exam Ref: 2082 Kartik, 2081 Chaitra, 2080 Ashwin, 2079 Chaitra, 2079 Jestha, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2074 Bhadra

**Q5. Differentiate between authentication and authorization. List and explain different types of security threats. How does OS manage these to provide security? [2+4+4]**

**Exam Ref:** 2080 Chaitra, 2079 Jestha, 2078 Chaitra

**Answer:**

### Authentication vs Authorization

| Feature | Authentication | Authorization |
|---|---|---|
| **Definition** | Verifies WHO you are (identity) | Determines WHAT you can do (permissions) |
| **Question Answered** | 'Are you who you claim to be?' | 'Are you allowed to do this?' |
| **Timing** | Happens FIRST (at login/entry) | Happens AFTER authentication |
| **Mechanism** | Passwords, biometrics, tokens, certificates | ACL, roles, permission bits, policies |
| **Example** | Username + Password login | User Alice can read but not write /etc/passwd |
| **Failure Result** | Access denied entirely | Specific operation denied (403 Forbidden) |

### Types of Authentication

- Something you KNOW: Password, PIN, security question
- Something you HAVE: Smart card, OTP token, security key (YubiKey)
- Something you ARE: Fingerprint, retina scan, facial recognition (biometrics)
- Two-Factor Authentication (2FA): Combines any two of the above — much more secure

**Password Security Best Practices:**

- Never store plaintext passwords — store salted hash (bcrypt, Argon2)
- Salt: Random value added before hashing to prevent rainbow table attacks
- Example: hash(password + salt) → stored hash

### OS Security Design Principles

| Principle | Explanation |
|---|---|
| **Least Privilege** | Every process/user should have only the minimum permissions needed to perform its function |
| **Fail-Safe Defaults** | Default state should be denial of access; access must be explicitly granted |
| **Economy of Mechanism** | Security mechanisms should be as simple as possible |
| **Complete Mediation** | Every access to every resource must be checked; no cached access decisions |

| | |
|---|---|
| **Open Design** | Security should not depend on secrecy of design — only on secrecy of keys |
| **Separation of Privilege** | Require multiple conditions to grant access (dual-key safes) |
| **Isolation** | Processes isolated from each other; kernel isolated from user space |

### Q6. What is a trapdoor? Explain the firewalls and Access Control List in security. [2+4]

**Exam Ref:** 2079 Ashwin, 2078 Chaitra

**Answer:**

## Trapdoor (Backdoor)

A trapdoor (or backdoor) is a secret method of bypassing normal authentication or access control in a system, intentionally left by a developer or maliciously inserted by an attacker.

- Legitimate use: Developers leave trapdoors for debugging or maintenance (should be removed before release)
- Malicious use: Attacker installs a backdoor after a successful intrusion to maintain access
- Example: The famous Ken Thompson (UNIX creator) described in 'Reflections on Trusting Trust' how a compiler could insert a trapdoor in any program, including the login command
- Detection: Code audits, binary integrity checking, intrusion detection systems

## Firewalls

A firewall is a network security device (hardware or software) that monitors and controls incoming and outgoing network traffic based on predetermined security rules. It establishes a barrier between a trusted internal network and untrusted external networks.

**Types of Firewalls:**

| Type | How It Works | Example |
|---|---|---|
| **Packet Filtering** | Examines IP headers (source/dest IP, port, protocol); blocks based on rules | iptables on Linux |
| **Stateful Inspection** | Tracks connection state; allows only packets that are part of valid established connections | Most modern firewalls |
| **Application Layer (Proxy)** | Inspects application-level data; understands HTTP, FTP, DNS | Squid proxy, WAF |
| **Next-Gen Firewall (NGFW)** | Deep packet inspection + IDS/IPS + application awareness | Palo Alto, Cisco Firepower |

## SECTION 4: ACL, Protection Matrix & OS Security Support

Exam Ref: 2082 Kartik, 2081 Chaitra, 2080 Ashwin, 2079 Chaitra, 2078 Chaitra, 2078 Baishakh, 2076 Baisakh

**Q7. Define Protection Domain and Protection Matrix. What design considerations must be taken into account to ensure security in a computer system? [4+4]**

**Exam Ref:** 2082 Kartik [4+4], 2079 Chaitra

**Answer:**

### Capability Lists vs Access Control Lists

| Feature | Capability List (C-List) | Access Control List (ACL) |
|---|---|---|
| **Organization** | Per PROCESS/DOMAIN (row of protection matrix) | Per OBJECT/RESOURCE (column of protection matrix) |
| **Content** | List of (Object, Rights) for each subject | List of (Subject, Rights) for each object |
| **Easy to Answer** | 'What can THIS process access?' | 'Who can access THIS file?' |
| **Revocation** | Hard — must find all capabilities | Easy — just modify the ACL entry |
| **Delegation** | Easy — process can pass capability to another | Hard — requires ACL modification |
| **Example** | Process P1: {(F1: rw), (F2: r), (P: print)} | File F1: {(P1: rw), (P2: r), (P3: none)} |

**UNIX Permission Model (Simplified ACL):**

```
ls -l /etc/passwd
-rw-r--r-- 1 root root 2847 Jan 1 12:00 /etc/passwd
 ^^^         = Owner (root): read+write
    ^^^      = Group (root): read only
       ^^^   = Others: read only
```

## QUICK REVISION: Chapter 8 — Key Points Summary

| Topic | Key Points | Expected Marks |
|---|---|---|
| **Types of Attacks** | Passive (eavesdrop) vs Active (modify); DoS, DDoS, Phishing, SQL Injection, Buffer Overflow, Trojan | 2–4 |
| **ACL** | List of (Subject, Rights) per resource; easy to revoke; hard to find what a user can access | 2–4 |
| **HASH / Message Digest** | One-way fixed-size output; SHA-256 used today; Avalanche effect; used for password storage + integrity | 2–3 |
| **Caesar Cipher** | Shift each letter by K positions; C=(P+K)mod26; Key K=3 example: HELLO→KHOOR | 2–4 |
| **Symmetric vs Asymmetric** | Symmetric: same key, fast; Asymmetric: public+private, slow but key exchange solved | 2–4 |
| **Public Key Cryptography** | Encrypt with receiver's public key; Sign with sender's private key; RSA algorithm | 3–5 |
| **Authentication vs Authorization** | Authentication=verify identity; Authorization=verify permissions; 2FA recommended | 2–3 |
| **Protection Domain & Matrix** | Domain = set of (Object, Rights); Protection Matrix: domains×objects grid | 3–5 |
| **Capability List vs ACL** | C-List per process (easy delegation); ACL per object (easy revocation) | 2–3 |
| **Firewall** | Packet filtering / Stateful / Application layer; controls traffic based on rules | 2–4 |
| **Trapdoor** | Hidden entry point bypassing security; must be removed before deployment | 2 |
| **OS Security Principles** | Least Privilege, Fail-Safe Defaults, Complete Mediation, Isolation | 2–3 |

**Best of Luck! | TU IOE BCT | CT 656 OS | Chapter 8: Security**